

Enhancing Genetic Algorithms: Adaptive Operators and Hybrid Local Search for Numerical Function Optimization

Olivia-Alexandra Gavril

30 November 2025

Abstract

This paper evaluates the performance of an optimized *Genetic Algorithm(GA)* in finding the minimum values for four standard test functions: De Jong, Schwefel, Rastrigin and Michalewicz. The study investigates three different problem sizes, or dimensions ($D = 5, 10$, and 30). Our primary goal is to compare the optimized GA's and the classic GA's results. Across all functions and dimensions, the improved GA consistently achieved near-optimal solutions, demonstrating strong robustness even for the high-dimensional case $D=30$. In contrast, the classical GA showed clear performance degradation for $D=30$, struggling to escape local optima and producing significantly weaker solutions. These findings indicate that the introduced enhancements substantially improve convergence reliability and solution quality.

1 Introduction

This paper presents a detailed evaluation of an improved Genetic Algorithm (GA) tested on four classical benchmark functions: De Jong, Schwefel, Rastrigin, and Michalewicz. These benchmark functions cover different levels of difficulty, ranging from smooth convex surfaces to highly multimodal and deceptive landscapes. This allows us to test the algorithm under varied and challenging conditions. The purpose of this study is to evaluate how the improved GA handles these problems and to see whether the added adaptive mechanisms and hybrid component actually lead to better optimization performance. To achieve this, the results of the improved GA are compared with those of a classical GA implementation, allowing us to clearly identify the benefits and limitations of the added enhancements.

Genetic algorithm was first proposed by John Holland motivated by Darwin's "Survival of fittest" principle. The algorithm can be viewed as an evolutionary process where a randomly selected population of individuals evolves throughout generations using three operators i.e. selection, crossover and mutation.[6] These mechanisms guide the search toward promising regions of the solution space while maintaining enough diversity to avoid stagnation. However, in the simple genetic algorithm there are primarily two reasons for premature convergence: insufficient genetic information in the initial population and loss of genetic information during optimization process.[6] When this happens, the population becomes uniform and the algorithm may converge to a suboptimal solution.

To address these limitations, this work incorporates adaptive mechanisms into the GA. The mutation and crossover probabilities are dynamically adjusted during the evolutionary process. *Rank-based adaptive mutation* assigns higher mutation rates to weaker individuals and lower rates to stronger ones, preserving high-quality solutions while encouraging broader exploration.[6] *Adaptive crossover* follows a similar principle: good individuals undergo crossover less frequently to avoid disruption, whereas poorer individuals are crossed more often, injecting new genetic material and improving population diversity.[7]

Finally, to further refine the best solution found by the GA, a Hill Climbing Best-Improvement procedure is applied as a local search step. This hybrid approach allows the algorithm to exploit the solution space more effectively and can significantly improve final solution quality.

2 Methods and Implementation

2.1 Solution Representation

In this work, each candidate solution is encoded as a bitstring. Since the benchmark functions operate on real-valued values, every variable within the interval $[a, b]$ is discretized according to a chosen precision

10^{-d} . This results in

$$N = (b - a) 10^d, \quad n = \lceil \log_2 N \rceil,$$

and the total chromosome length becomes $L = n \times D$. [1]

Instead of using classical binary encoding, each variable is represented using **Gray code**, a binary numbering system in which consecutive values differ by exactly one bit. [2]

The Gray encoding of a binary number $b = b_{n-1} \dots b_0$ is defined as:

$$g_0 = b_0, \quad g_i = b_i \oplus b_{i-1}, \quad i = 1, \dots, n-1,$$

where $\oplus$ denotes the XOR operation. [2]

Decoding applies the inverse transformation:

$$b_0 = g_0, \quad b_i = b_{i-1} \oplus g_i, \quad i = 1, \dots, n-1.$$

After decoding to the integer v , the value is scaled back into the original interval $[a, b]$ using a straightforward linear conversion.

$$X_{\text{real}} = a + \frac{v(b - a)}{2^n - 1}.$$

[1]

In natural binary representation, multiple bits can change simultaneously between two successive numbers, potentially causing errors in digital systems. In contrast, Gray code is designed so only a single bit changes between successive values, reducing the risk of errors during transitions. [3]

2.2 Genetic Algorithm

Genetic Algorithms are optimization methods that work with a population of possible solutions, each encoded as a binary chromosome. A fitness function evaluates how good each solution is, and the population improves over generations through selection, crossover, mutation, and elitism. Together, these steps gradually guide the search toward better solutions.

Selection

Selection determines which individuals contribute to the next generation and is one of the key mechanisms driving evolutionary progress. In this work, we apply a *Roulette Wheel Selection*, a fitness-proportional method commonly used in genetic algorithms. The central idea is to interpret the population as a probability wheel: each individual receives a segment whose size is proportional to its fitness value.[1] Individuals with better fitness occupy larger segments, increasing their likelihood of being chosen as parents. However, weaker individuals still retain a non-zero probability of selection, which helps maintain genetic diversity and prevents the algorithm from prematurely converging.

Crossover and Adaptive Crossover Probability

Crossover is the principal search operator in genetic algorithms. By combining genetic material from two parent chromosomes, it generates offspring that inherit complementary characteristics from both sources. Crossover adds variation and spreads good traits through the population, helping the algorithm focus on better regions of the search space.

In classical implementations, the crossover probability p_c is fixed throughout the evolution. However, applying the same crossover rate to all individual, regardless of their fitness, can lead to undesirable effects. High-quality solutions may be disrupted when the population is close to convergence, while insufficient variation may be introduced when the algorithm becomes trapped near a local optimum.

To address this limitation, we employ an *adaptive* crossover scheme. Convergence is estimated through the difference between the maximum fitness $f_{\max}$ and the population average $\bar{f}$, with a small value of $f_{\max} - \bar{f}$ indicating that the population has become homogeneous [7]. Let f' denote the *larger* fitness value of the two parent solutions selected for crossover. The probability of performing crossover is then defined as:

$$p_c = \begin{cases} k_1 \frac{f_{\max} - f'}{f_{\max} - \bar{f}}, & f' \geq \bar{f}, \\ k_2, & f' < \bar{f}, \end{cases} \quad (5)$$

[7]

where $k_1, k_2 \leq 1.0$ are constants, typically chosen as [7] :

$$k_1 = 1.0, \quad k_2 = 0.5.$$

Rank-Based Adaptive Mutation

Mutation is the main source of random variation in a genetic algorithm. By flipping bits or altering genes with a small probability, it helps the population escape local optima and explore new regions of the search space. However, applying the same mutation rate to all individuals can either damage high-quality solutions or fail to sufficiently diversify poor ones.

This motivates the use of an *adaptive, rank-based mutation scheme*. Instead of using raw fitness values, individuals are first sorted according to fitness and assigned ranks from 1 (worst) to N (best). The mutation probability then decreases linearly with rank, ensuring that weak solutions explore more aggressively while strong solutions are preserved. [6]

The mutation probability for an individual of rank r is given by:

$$p = p_{\max} \left(1 - \frac{r - 1}{N - 1} \right), \quad (1)$$

[6] where:

- p is the mutation probability for the chromosome,
- $p_{\max}$ is the maximum allowed mutation probability,
- r is the rank of the individual ($1 = \text{worst}$, $N = \text{best}$),
- N is the population size. [6]

Elitism

Elitism helps the algorithm keep the best solutions by copying a small number of top individuals directly into the next generation, without applying crossover or mutation to them. This prevents the best solution found so far from being lost due to randomness and ensures that the population's best fitness does not decrease. [4]

3 Experiments and Results

Experimental Setup

All benchmark functions were optimized 30 times under three problem dimensionalities ($D = 5, 10, 30$), using a binary encoding scheme with a fixed precision of five decimal places. The algorithm adapts its parameter configuration depending on the problem size. For lower-dimensional problems, a smaller population and fewer generations are sufficient, while higher-dimensional problems require larger populations, more iterations, and slightly higher mutation rates to maintain diversity.

- **Population size:** 100 (all functions for $D = 5, 10$), 300 (Schwefel, $D = 30$), 250 (Rastrigin, $D = 30$), 200 (De Jong, Michalewicz, $D = 30$);
- **Crossover rate:** 0.85 (all functions for $D = 5, 10, 30$);
- **Mutation rate:** 0.015 (all functions for $D = 5, 10$), 0.03 (Schwefel, $D = 30$), 0.025 (Rastrigin, $D = 30$), 0.02 (De Jong, Michalewicz, $D = 30$);
- **Maximum generations:** 1000 (all functions for $D = 5, 10$), 3000 (Schwefel, $D = 30$), 3500 (Rastrigin, $D = 30$), 2000 (De Jong, Michalewicz, $D = 30$).

Elitism is set to 10% of the population for lower dimensions ($D = 5, 10$), but reduced to 5% for difficult high-dimensional ($D = 30$) functions. The **Hill Climbing** procedure used was limited to a maximum of *100 iterations*. Because of the high computational time of the classical GA, we included **parallelization** in the implementation, which reduced the running time significantly.

3.1 De Jong's Function

So called *first function of De Jong's* is one of the simplest test benchmark. [8] The function is continuous, convex and unimodal. General function definition and restrictions:

$$f_1(\mathbf{x}) = \sum_{i=1}^n x_i^2, \quad -5.12 \leq x_i \leq 5.12 \quad (2)$$

Global minimum $f(x) = 0$ is obtainable for $x_i = 0$, where $i = 1, \dots, n$. [9]

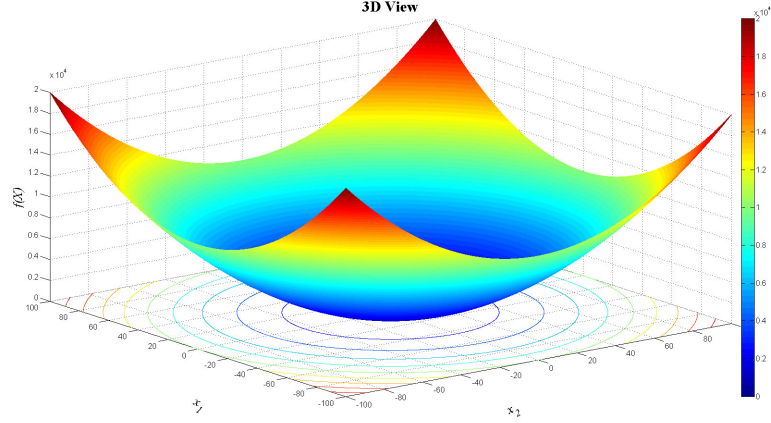


Figure 1: Visualisation of De Jong's function, with a surface plot for variables in the range $[-500, 500]$.

Optimized GA	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	0.00000	0.00000	0.00000	0.00000
Dimension = 10	0.00000	0.00000	0.00000	0.00000
Dimension = 30	0.00000	0.00000	0.00000	0.00000

Table 1: Statistics for different dimensions using the optimized Genetic Algorithm on De Jong's function.

Genetic Algorithm	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	0.00000	0.00000	0.00000	0.00000
Dimension = 10	0.00000	0.00000	0.00000	0.00000
Dimension = 30	0.00001	0.00006	0.00003	0.00001

Table 2: Statistics for different dimensions using the Genetic Algorithm on De Jongs function.

Optimized GA Running Time	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	3.89225	8.79240	5.08654	1.26568
Dimension = 10	12.96130	16.62995	14.78796	1.14959
Dimension = 30	103.56974	211.35959	177.2656	24.30799

Table 3: Statistics for different dimensions using the optimized Genetic Algorithm on De Jong's function.

GA Running Time	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	14.64682	20.42869	16.15386	1.27019
Dimension = 10	24.55145	28.15953	27.22605	0.73739
Dimension = 30	276.93575	283.87172	278.5448	1.16416

Table 4: Statistics for the running time of the Genetic Algorithm(GA) on De Jongs function, in seconds.

Both algorithms perform well on De Jong’s function and reach the minimum, but the running time for the optimized GA is better. There is a slight difference at dimension 30, where the classic GA starts to be weaker, but overall the algorithms exhibit high performance.

3.2 Schwefel’s Function

Schwefel’s function is particularly deceptive because its global minimum is located far away from most local minima, making it challenging for optimization algorithms to locate the true optimum. The function has the following definition and restrictions:

$$f_7(\mathbf{x}) = \sum_{i=1}^n -x_i \cdot \sin\left(\sqrt{|x_i|}\right), \quad -500 \leq x_i \leq 500 \quad (3)$$

Its global minimum $f(x) = -418.9829n$ is obtainable for $x_i = 420.9687$, where $i = 1, \dots, n$. [9]

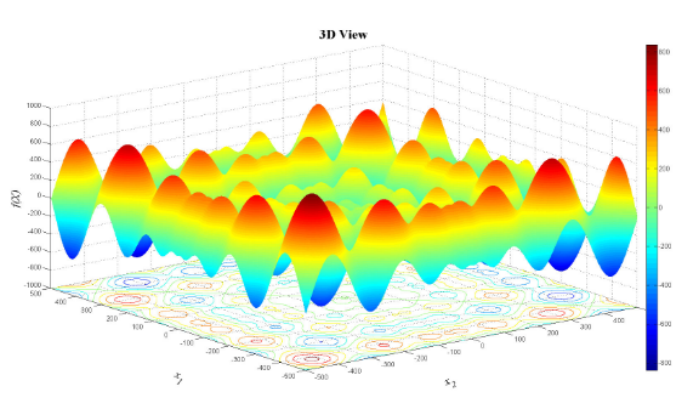


Figure 2: Visualization of Schwefel’s function with a surface plot for variables in the range $[-500, 500]$.

Optimized GA	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	-2094.91444	-2094.91444	-2094.91444	0.00000
Dimension = 10	-4189.82887	-4189.82887	-4189.82887	0.00000
Dimension = 30	-12569.48662	-12569.48662	-12569.48662	0.00000

Table 5: Statistics for different dimensions using the optimized Genetic Algorithm on Schwefel’s function.

Genetic Algorithm	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	-2094.91444	-2094.70648	-2094.89	0.05221
Dimension = 10	-4189.82630	-4188.99778	-4189.415	0.20996
Dimension = 30	-12392.81396	-11537.29631	-12056.95	176.7721

Table 6: Statistics for different dimensions using the Genetic Algorithm on Schwefel’s function.

Optimized GA Running Time	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	5.78833	12.01960	9.84781	1.14123
Dimension = 10	9.62999	28.14897	16.29459	5.70735
Dimension = 30	332.65087	881.01340	588.0256	125.3727

Table 7: Statistics for different dimensions using the optimized Genetic Algorithm on Schwefel's function.

GA running time	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	28.34499	35.36355	28.94331	1.34845
Dimension = 10	44.54907	45.35387	44.89211	0.18126
Dimension = 30	403.50527	407.87313	405.5833	0.96331

Table 8: Statistics for the running time of the Genetic Algorithm(GA) on Schwefel's function, in seconds.

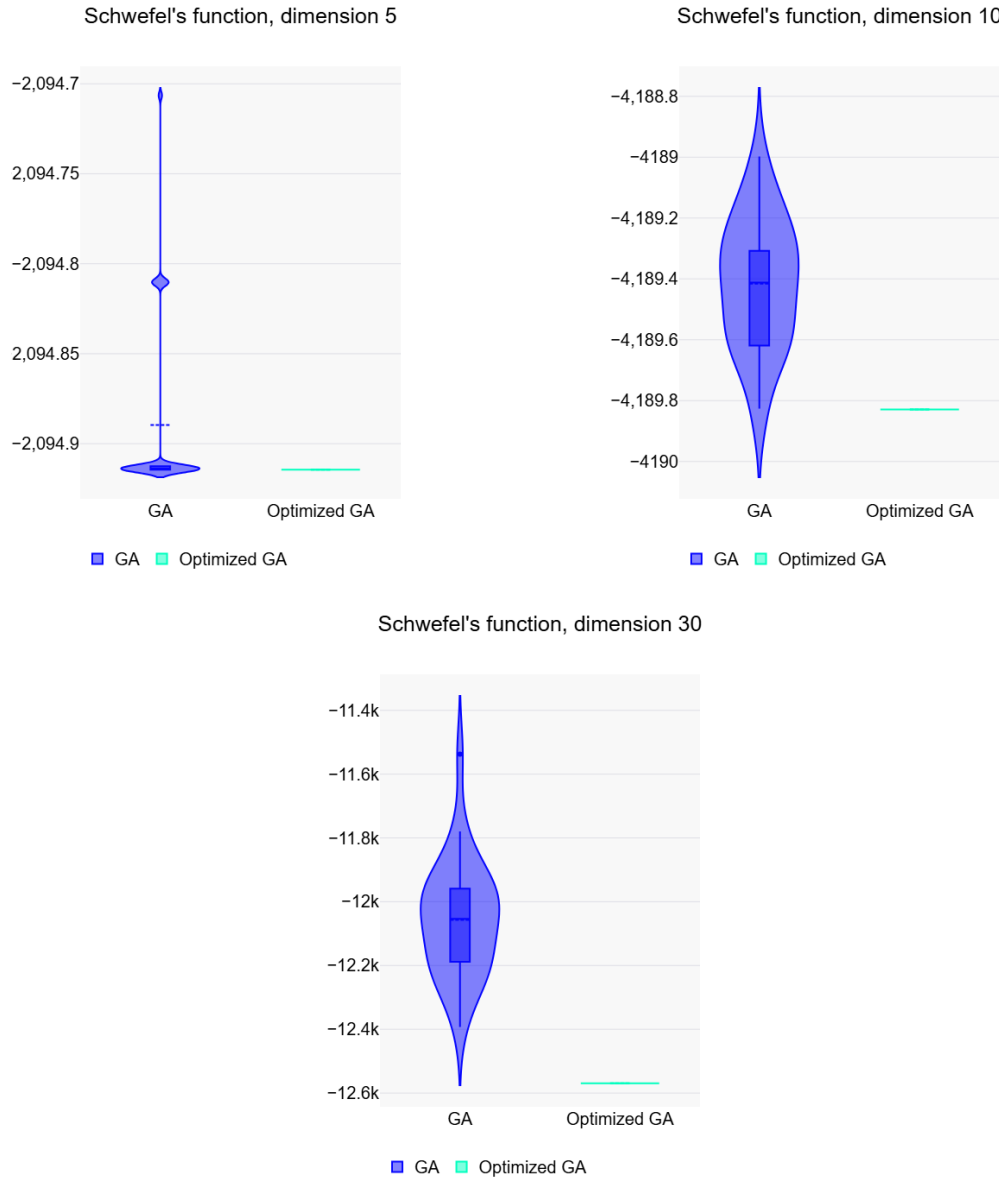


Figure 3: Violin plots for the Genetic Algorithms tested on Schwefel's function, on different dimensions.

The optimized GA manages to reach local minima in every run, all standard deviations being equal to 0. While the classic GA is not far from the optimum, it performs worse, especially in larger dimensions, where the standard deviation is much higher.

3.3 Rastrigin's Function

Rastrigin's function is based on the function of De Jong with the addition of cosine modulation in order to produce frequent local minima. Therefore, the test function is highly multimodal. However, the location of the minima are regularly distributed.[9] The function has the following definition and constrains:

$$f_6(\mathbf{x}) = 10 \cdot n + \sum_{i=1}^n (x_i^2 - 10 \cdot \cos(2\pi x_i)), \quad -5.12 \leq x_i \leq 5.12 \quad (4)$$

Global minimum $f(x) = 0$ is obtainable for $x_i = 0$, where $i = 1, \dots, n$. [9]

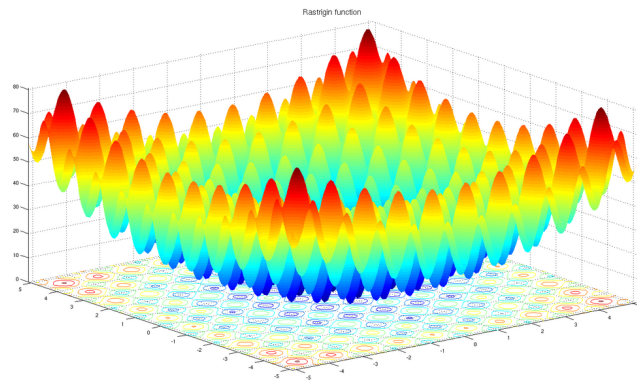


Figure 4: Visualization of Rastrigin's function left with a surface plot for variables in the range $[-5, 5]$.

Optimized GA	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	0.00000	0.00000	0.00000	0.00000
Dimension = 10	0.00000	3.97983	0.13266	0.72661
Dimension = 30	0.00000	0.99496	0.16582	0.37713

Table 9: Statistics for different dimensions using the optimized Genetic Algorithm on Rastrigin's function.

Genetic Algorithm	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	0.00000	0.00000	0.00000	0.00000
Dimension = 10	0.00016	3.50270	0.48901	0.92920
Dimension = 30	23.77464	50.66660	33.26112	6.06273

Table 10: Statistics for different dimensions using the Genetic Algorithm on Rastrigin's function.

Optimized GA Running Time	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	4.94872	5.88020	5.08724	0.20289
Dimension = 10	7.96408	16.47161	10.20085	2.90795
Dimension = 30	242.14497	537.87200	372.2509	94.44029

Table 11: Statistics for different dimensions using the optimized Genetic Algorithm on Rastrigin's function.

GA Running Time	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	21.98238	39.50247	27.96103	3.96365
Dimension = 10	33.82661	39.58252	34.74178	1.39452
Dimension = 30	1279.6	3119.3	2008.167	326.7985

Table 12: Statistics for the running time of the Genetic Algorithm(GA) on Rastrigin's function, in seconds.

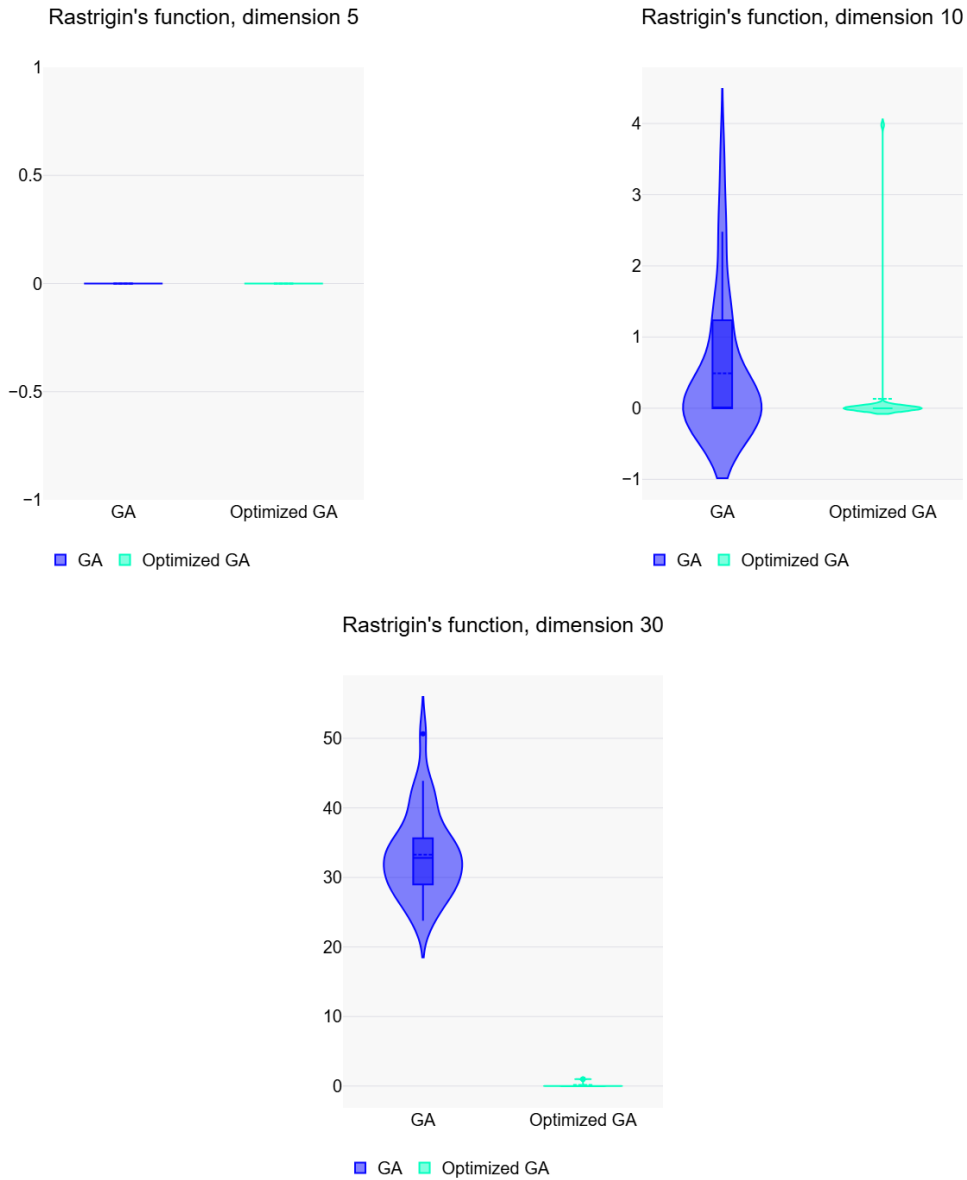


Figure 5: Violin plots for the Genetic Algorithms tested on Rastrigin's function, on different dimensions.

For dimensions 5 and 10, both algorithms perform well, with the optimized GA being slightly better at dimension 10. At dimension 30, the classic GA starts to get weaker, failing to reach the minimum, unlike the optimized GA which still achieves optimal results.

3.4 Michalewicz's Function

The Michalewicz function is a multimodal benchmark function that possesses $n!$ local optima. [9] The parameter m controls the steepness of the valleys and ridges: higher values of m make the optimization problem increasingly difficult. For large m values, the function exhibits "needle-in-a-haystack" behavior, where most points in the search space provide little information about the location of the global optimum. [8]

$$f_{12}(\mathbf{x}) = - \sum_{i=1}^n \sin(x_i) \left[\sin \left(\frac{i x_i^2}{\pi} \right) \right]^{2m}, \quad i = 1, \dots, n, \quad m = 10, \quad 0 \leq x_i \leq \pi \quad (5)$$

The global minimum value was approximated as $f(x) = -4.687$ for $n = 5$ and $f(x) = -9.66$ for $n = 10$. The corresponding optimal solutions were not provided. [9] [8]

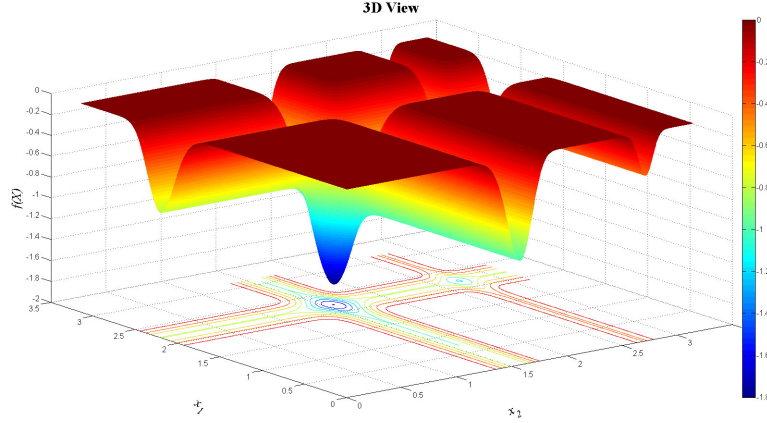


Figure 6: Visualization of the Michalewicz's function. Surface plot for variables x_3 and x_4 over the interval $[0, 3]$, and surface plot for variables x_1 and x_2 over the interval $[0, 3]$.

Optimized GA	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	-4.68766	-4.37490	-4.62539	0.08866
Dimension = 10	-9.62030	-9.04140	-9.38991	0.14883
Dimension = 30	-29.03670	-27.07703	-28.48711	0.38304

Table 13: Statistics for different dimensions using the optimized Genetic Algorithm on Michalewicz's function.

Genetic Algorithm	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	-4.68766	-4.68593	-4.68754	0.00043
Dimension = 10	-9.65445	-9.32431	-9.50911	0.10053
Dimension = 30	-25.03028	-22.38157	-23.58379	0.54776

Table 14: Statistics for different dimensions using the Genetic Algorithm on Michalewicz's Function.

Optimized GA Running Time	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	4.80553	8.73114	5.98211	0.96556
Dimension = 10	7.40796	10.60143	7.98563	0.88259
Dimension = 30	276.43135	309.55296	292.7886	6.97667

Table 15: Statistics for different dimensions using the optimized Genetic Algorithm on Michalewicz's function.

GA Running Time	Min. Value	Max. Value	Mean	Standard Deviation
Dimension = 5	22.17795	22.83910	22.39582	0.18817
Dimension = 10	36.22302	37.28835	36.55281	0.2429
Dimension = 30	149.27843	151.05406	149.9398	0.37358

Table 16: Statistics for the running time of the Genetic Algorithm on Michalewicz's Function, in seconds.

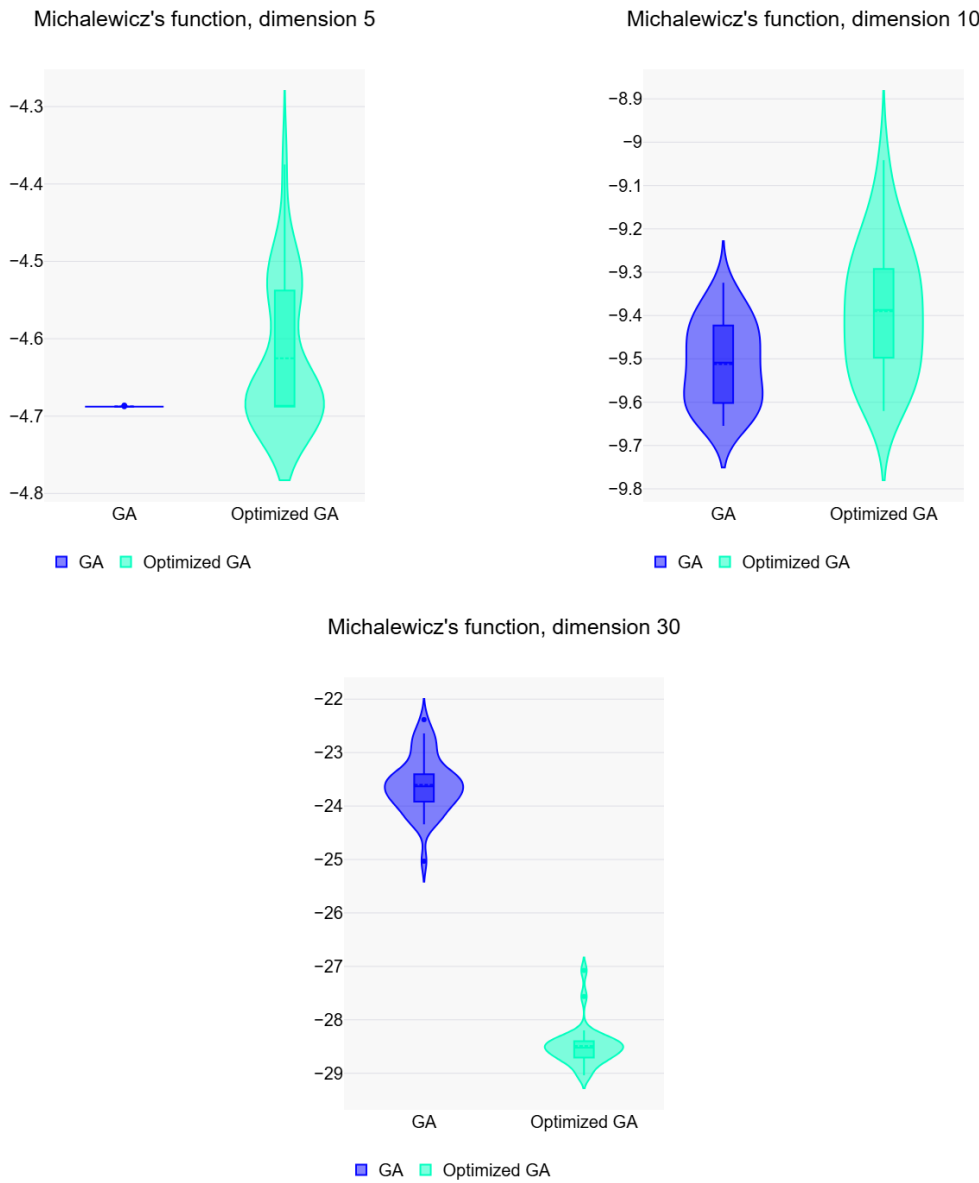


Figure 7: Violin plots for the Genetic Algorithms tested on Michalewicz's function, on different dimensions.

The optimized GA reaches values close to minimum at dimension 30, while the classic GA stays much further away. This shows that the adaptive operators and the hill-climbing step clearly improve performance on this highly deceptive benchmark. For smaller dimensions, the optimized GA shows slightly more variation than the classical GA. This happens because the adaptive operators encourage more exploration, so the algorithm tries more search directions. In contrast, the simple GA tends to converge prematurely, which results in lower variance but worse overall performance.

Global Performance Insights

For low-dimensional settings and functions with smoother landscapes, both algorithms deliver excellent performance: the distributions are narrow, the variance is minimal, and the best values cluster tightly around the global optimum. While the classic GA can still have more variations, the optimized GA almost always reaches the minima, with visible results being at the Schwefel’s function. As dimensionality increases, the quality of the classic GA’s solutions decreases noticeably. The violin plots broaden and the statistical variance grows, indicating a decline in convergence stability. The optimized GA, however, still manages to achieve great performance and optimal results, being more stable even at higher dimensions and demonstrating that the adaptive operators and hill-climbing refinement successfully preserve solution quality.

4 Conclusions

This study demonstrates that adaptive mechanisms and hybrid local search substantially enhance the classic Genetic Algorithm performance, particularly for high-dimensional optimization problems. The optimized GA consistently achieved near-optimal solutions across all four benchmark functions and three dimensionalities tested, with the most pronounced improvements observed at dimension 30. The rank-based adaptive mutation effectively balanced exploration and exploitation by encouraging diversity among weak solutions while preserving elite individuals. Similarly, adaptive crossover prevented premature convergence by dynamically adjusting recombination rates based on population fitness distribution. The hill-climbing refinement step provided the final advantage, polishing solutions to reach precise optima. Overall, these methods confirm that thoughtful integration of adaptive operators and hybridization strategies can significantly improve GA effectiveness for optimization problems.

Resources and references

- [1] Lect. dr. Eugen Croitoru. *Genetic Algorithms*.
Available at: <https://profs.info.uaic.ro/eugen.croitoru/teaching/ga/>
- [2] GeeksforGeeks."Gray to Binary and Binary to Gray conversion."(2025, October 3).Available at:<https://www.geeksforgeeks.org/dsa/gray-to-binary-and-binary-to-gray-conversion/>
- [3] Francesco Galgani."Gray Code vs Natural Binary Representation".Baeldung on Computer Science.(2024 May 25). Available at:<https://www.baeldung.com/cs/gray-code-vs-base-two-representation>
- [4] Chris Woody Woodruff, "Genetic Algorithms" *Elitism for Evolution Survival of the Fittest*. Fractional architect.(2025, June 26). Available at: <https://www.woodruff.dev/day-12-genetic-algorithms-elitism-for-evolution-survival-of-the-fittest>
- [5] Uday K. Chakraborty, Cezary Z. Janikow, "An analysis of Gray versus binary encoding in genetic search", Information Sciences, Volume 156, Issues 3–4, 2003, Pages 253-269, ISSN 0020-0255. Available at: (<https://www.sciencedirect.com/science/article/pii/S0020025503001786>)
- [6] Avijit Basak, IBM India Pvt. Ltd., Kolkata, India, "A Rank based Adaptive Mutation in Genetic Algorithm". International Journal of Computer Applications (0975 – 8887) Volume 175– No. 10, August 2020. Available at: <https://arxiv.org/pdf/2104.08842>
- [7] M. Srinivas, and L. M. Patnaik, Fellow, ZEEE "Adaptive Probabilities of Crossover and Mutation in Genetic Algorithms" , IEEE TRANSACTIONS ON SYSTEMS, MAN AND CYBERNETICS, VOL. 24, NO. 4, APRIL 1994. Available at: https://wiki.eecs.yorku.ca/course_archive/2013-14/F/4403/_media/ga2.pdf .
- [8] Marcin Molga, Czesław Smutnicki. *Test functions for optimization needs*. Wrocław University of Technology(3 april 2005)
- [9] *GEATBX: Examples of Objective Functions*.
Genetic and Evolutionary Algorithm Toolbox for use with Matlab. 1994-2000 Hartmut Pohlheim.
Available at: <http://www.geatbx.com/docu/fcnindex-01.html> .
- [10] Berkeley Statistics,."Parallel processing in C/C++." Available at: <https://computing.stat.berkeley.edu/tutorial-parallelization/parallel-C.html>
- [11] GeeksforGeeks, "Basic concepts of optimizing for parallelism and locality", (2025, July 23). Available at: <https://www.geeksforgeeks.org/compiler-design/basic-concepts-of-optimizing-for-parallelism-and-locality/>